

TP Web Services

Daniel Hagimont (daniel.hagimont@irit.fr)

The objective of this lab is to introduce the use of Web Services (WS). We will be working with WS in REST mode.

0) Installs

I assume that vscode and java (JDK 17) are properly installed.

To install Tomcat:

```
cd
```

```
mkdir ws
```

```
cd ws
```

```
wget https://sd-160040.dedibox.fr/hagimont/software/apache-tomcat-11.0.1.tar.gz
```

```
tar xzf apache-tomcat-11.0.1.tar.gz
```

```
# edit your .bashrc and add
```

```
export TOMCAT_HOME=$HOME/ws/apache-tomcat-11.0.1
```

```
export PATH=$TOMCAT_HOME/bin:$PATH
```

Create a new terminal to take into account the modifications in your .bashrc

To start or stop tomcat from a terminal:

```
startup.sh
```

```
shutdown.sh
```

Under vscode, to use Spring-boot :

install this extension: spring initializr java support

1) Creating a WS with Spring-boot

A REST WS has been implemented. It is deployed on a machine in the cloud and is available at the following address:

<http://sd-160040.dedibox.fr:8080/students-server>

It provides two interfaces:

Method **getstudent**, parameters **firstname** and **lastname**, returns a **Json**.

Call example:

<http://sd-160040.dedibox.fr:8080/students-server/getstudent?firstname=Alain&lastname=Tchana>

```
{
  "firstname": "Alain",
  "lastname": "Tchana",
  "birthdate": "18/12/1984",
  "sex": "male",
  "address": "3 rue Jeff Rouchon",
  "city": "Toulouse",
  "zip": "31000",
  "country": "France",
  "phone": "0102030405",
```

```
"email": "alain.tchana@enseeiht.fr",  
"ine": "1111111111"
```

```
}
```

Method **getrecord**, parameter **ine**, returns a **Json**.

Call example:

<http://sd-160040.dedibox.fr:8080/students-server/getrecord?ine=1111111111>

```
{
```

```
  "mathematics": "12",  
  "middleware": "14",  
  "networks": "11",  
  "systems": "5",  
  "architecture": "16",  
  "programming": "18",  
  "ine": "1111111111"
```

```
}
```

Test this online WS with your web browser.

Recreate this project with spring-boot in vscode. The sources of the classes are given in the students-server-src folder.

Create Spring-boot project:

View -> Command Palette

-> Spring initializ : Create a Maven Project

-> 3.4.1 -> Java -> n7 -> students-server -> War -> 17

-> Selected 0 dependencies

Select the location where the project is created. Then, open the created project.

Copy classes Student, Record and StudentsController in src/main/java/n7/students_server/

Execute and test this project:

- as a standalone application
 - in the project: ./mvnw spring-boot:run
 - it starts your application in a tomcat server
 - URL: localhost:8080/getstudent....
 - NB : in this mode, you don't have the project name in the URL
- by exporting a war in a running tomcat
 - in the project: ./mvnw package
 - the generated war is located in the target folder
 - start your tomcat (startup.sh)
 - copy the war with the students-server.war name in the webapp folder of your tomcat
 - URL: localhost:8080/students-server/getstudent....
 - NB : in this mode, you have the project name in the URL

You can rely on the slides from the WS lecture.

2) Calling a WS with RestEasy

Implement in a Java project a client program that invokes the previously mentioned service (using reeasy). You need to:

- download reeasy :

<https://sd-160040.dedibox.fr/hagimont/software/reteasy-jaxrs-3.0.9.Final-all.zip>

- create a Java project (students-client) in vscode
- include in your java project (referenced libraries) the reeasy jars (in lib)
- create the Java beans associated with the used Json (Student, Record)
- describe the service interface with annotations @GET, @Path, @Produces ...
- create and use the reeasy proxy
- an exemple is detailed in the slides from the WS lecture.
- we provide a script (run.sh) to execute the reeasy client (update it if necessary), or you can run the application in vscode

3) Develop a new WS

Create a REST WS (named students-marks) which returns, from the firstname/lastname of a student and the name of a lecture, the mark that this student obtained in that discipline. This WS is both client (from the previous WS, students-server) and server as it provides an interface:

Method **getmark**, parameters **firstname**, **lastname** and **lecture**, returns an **Integer**.

Call example:

http://localhost:8080/students-marks/getmark?
firstname=Alain&lastname=Tchana&lecture=systems

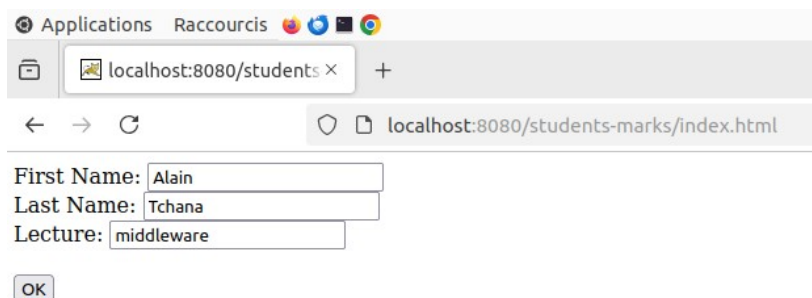
5

To implement this WS, you have to create a new Spring-boot project. You can refer to the project provided in 1) for guidance

To invoke service students-server from service students-marks, we will use the RestClient API from Spring. An example is detailed in the slides from the WS lecture.

Implement this WS and test it with a web browser.

Another way to test your WS is to use a little web page (that we provides, index.html) which includes a JavaScript code sequence which invokes this REST API. Install this web page in the students-marks project (in src/main/resources/static) and test it with this page.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/students-marks/index.html'. Below the address bar, there is a form with three input fields: 'First Name:' with the value 'Alain', 'Last Name:' with the value 'Tchana', and 'Lecture:' with the value 'middleware'. An 'OK' button is located below the form fields.

```

1 // Partie commune
2
3 public class Record {
4     String INE;    String mathematics;    String middleware;    String networks;
5     String systems;    String architecture;    String programming;
6
7     public Record() {}
8
9     public Record(String INE, String mathematics, String middleware, String networks,
10        String systems, String architecture, String programming) {
11         super();
12         this.INE = INE;
13         this.mathematics = mathematics;
14         this.middleware = middleware;
15         this.networks = networks;
16         this.systems = systems;
17         this.architecture = architecture;
18         this.programming = programming;
19     }
20
21     public String getINE() { return INE; }
22     public void setINE(String INE) { this.INE = INE; }
23     public String getMathematics() { return mathematics; }
24     public void setMathematics(String mathematics) { this.mathematics = mathematics; }
25     public String getMiddleware() { return middleware; }
26     public void setMiddleware(String middleware) { this.middleware = middleware; }
27     public String getNetworks() { return networks; }
28     public void setNetworks(String networks) { this.networks = networks; }
29     public String getSystems() { return systems; }
30     public void setSystems(String systems) { this.systems = systems; }
31     public String getArchitecture() { return architecture; }
32     public void setArchitecture(String architecture) { this.architecture = architecture; }
33     public String getProgramming() { return programming; }
34     public void setProgramming(String programming) { this.programming = programming; }
35 }
36
37 public class Student {
38     String INE;    String firstname;    String lastname;    String birthdate;
39     String sex;    String address;    String city;    String zip;
40     String country;    String phone;    String email;
41
42     public Student() {}
43
44     public Student(String INE, String firstname, String lastname,
45        String birthdate, String sex, String address, String city,
46        String zip, String country, String phone, String email) {
47         super();
48         this.INE = INE;
49         this.firstname = firstname;
50         this.lastname = lastname;
51         this.birthdate = birthdate;
52         this.sex = sex;
53         this.address = address;

```

```

54     this.city = city;
55     this.zip = zip;
56     this.country = country;
57     this.phone = phone;
58     this.email = email;
59 }
60
61 public String getINE() { return INE; }
62 public void setINE(String INE) { this.INE = INE; }
63 public String getFirstname() { return firstname; }
64 public void setFirstname(String firstname) { this.firstname = firstname; }
65 public String getLastname() { return lastname; }
66 public void setLastname(String lastname) { this.lastname = lastname; }
67 public String getBirthdate() { return birthdate; }
68 public void setBirthdate(String birthdate) { this.birthdate = birthdate; }
69 public String getSex() { return sex; }
70 public void setSex(String sex) { this.sex = sex; }
71 public String getAddress() { return address; }
72 public void setAddress(String address) { this.address = address; }
73 public String getCity() { return city; }
74 public void setCity(String city) { this.city = city; }
75 public String getZip() { return zip; }
76 public void setZip(String zip) { this.zip = zip; }
77 public String getCountry() { return country; }
78 public void setCountry(String country) { this.country = country; }
79 public String getPhone() { return phone; }
80 public void setPhone(String phone) { this.phone = phone; }
81 public String getEmail() { return email; }
82 public void setEmail(String email) { this.email = email; }
83 }
84
85 // Partie client
86
87 import jakarta.ws.rs.GET;
88 import jakarta.ws.rs.Produces;
89 import jakarta.ws.rs.QueryParam;
90 import jakarta.ws.rs.Path;
91
92 @Path("/")
93 public interface ServiceInterface {
94     @GET
95     @Path("/getstudent")
96     @Produces({"application/json"})
97     public Student getStudent(@QueryParam("firstname") String firstName,
98                               @QueryParam("lastname") String lastName);
99
100     @GET
101     @Path("/getrecord")
102     @Produces({"application/json"})
103     public Record getRecord(@QueryParam("ine") String ine);
104 }
105
106 import jakarta.ws.rs.client.ClientBuilder;
107 import jakarta.ws.rs.core.UriBuilder;

```

```

108 import org.jboss.resteasy.client.jaxrs.ResteasyClient;
109 import org.jboss.resteasy.client.jaxrs.ResteasyClientBuilder;
110 import org.jboss.resteasy.client.jaxrs.ResteasyWebTarget;
111 import org.jboss.resteasy.client.jaxrs.internal.ResteasyClientBuilderImpl;
112
113 public class TestClient {
114     public static final String url = "http://localhost:8080";
115     private static final String studFirstname = "Alain";
116     private static final String studName = "Tchana";
117
118     public static void main(String[] args) {
119         ResteasyClient client = new ResteasyClientBuilderImpl().build();
120         ResteasyWebTarget target = client.target(UriBuilder.fromPath(url));
121         ServiceInterface proxy = target.proxy(ServiceInterface.class);
122         Student student = proxy.getStudent(studFirstname, studName);
123         Record record = proxy.getRecord(student.getINE());
124         System.out.println("middleware grade of " + studFirstname + " "
125                             + studName + " = " + record.getMiddleware());
126     }
127 }
128
129 // Partie serveur
130
131 package n7.students_server;
132 import org.springframework.boot.builder.SpringApplicationBuilder;
133 import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;
134
135 public class ServletInitializer extends SpringBootServletInitializer {
136     @Override
137     protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
138         return application.sources(StudentsServerApplication.class);
139     }
140 }
141
142 package n7.students_server;
143 import java.util.Hashtable;
144 import org.springframework.web.bind.annotation.GetMapping;
145 import org.springframework.web.bind.annotation.RequestParam;
146 import org.springframework.web.bind.annotation.RestController;
147
148 @RestController
149 public class StudentsController {
150     static Hashtable<String, Student> students = new Hashtable<String, Student>();
151     static Hashtable<String, Record> records = new Hashtable<String, Record>();
152
153     static void addStudent(Student s) {
154         students.put(s.getFirstname() + "-" + s.getLastname(), s);
155     }
156
157     static void addRecord(Record r) {
158         records.put(r.getINE(), r);
159     }
160
161     @GetMapping("/getstudent")

```

```

162     public Student getStudent(@RequestParam("firstname") String firstname,
163                               @RequestParam("lastname") String lastname) {
164         String key = firstname + "-" + lastname;
165         System.out.println("access student: " + key);
166         return students.get(key);
167     }
168
169     @GetMapping("/getrecord")
170     public Record getRecord(@RequestParam("ine") String ine) {
171         System.out.println("access record: " + ine);
172         return records.get(ine);
173     }
174
175     // Static initializer : bloc run une seule fois avant que le main soit appelé
176     static {
177         addStudent(new Student(...));
178         // Plusieurs addStudent ici avec des données fictives
179         addRecord(new Record(...));
180         // Plusieurs addRecord ici avec des données fictives
181     }
182 }
183
184 package n7.students_server;
185 import org.springframework.boot.SpringApplication;
186 import org.springframework.boot.autoconfigure.SpringBootApplication;
187
188 @SpringBootApplication
189 public class StudentsServerApplication {
190     public static void main(String[] args) {
191         SpringApplication.run(StudentsServerApplication.class, args);
192     }
193 }

```